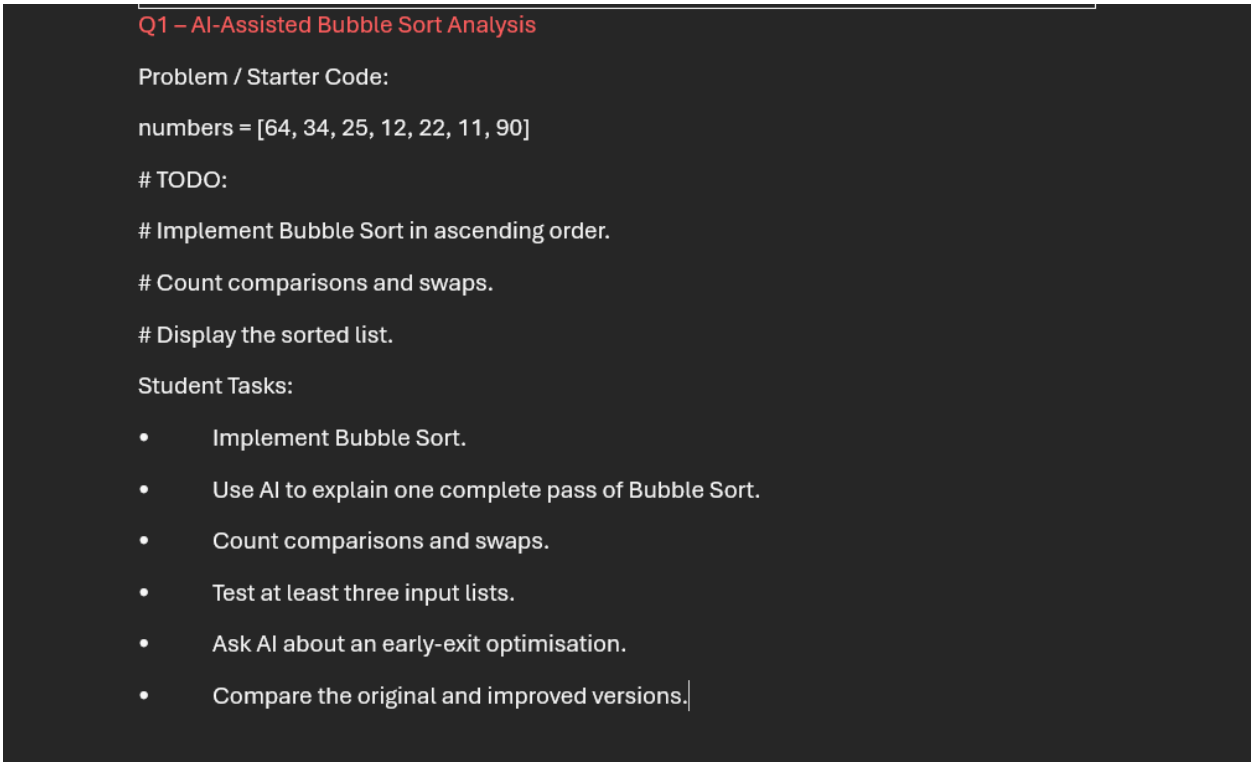
	SR UNIVERSITY Campus Warangal
	Program: II - B. Tech (CS& AI) Batch:
	Name: B.Spoorthi Reg. No:2503A52439
	Department: Computer Science and AI Semester: I
	AI Assisted Coding-Assignment Date: 22-09-2026



PROMPT:

Suggested AI Prompt:

“Explain Bubble Sort step by step for the given list. Identify opportunities to reduce unnecessary comparisons. Do not write the complete program yet.”

CODE:

```python
numbers = [64, 34, 25, 12, 22, 11, 90]


def bubble_sort(numbers):
    """Sort a list using Bubble Sort and return the result and counters."""
    sorted_numbers = numbers.copy()
    comparisons = 0
    swaps = 0

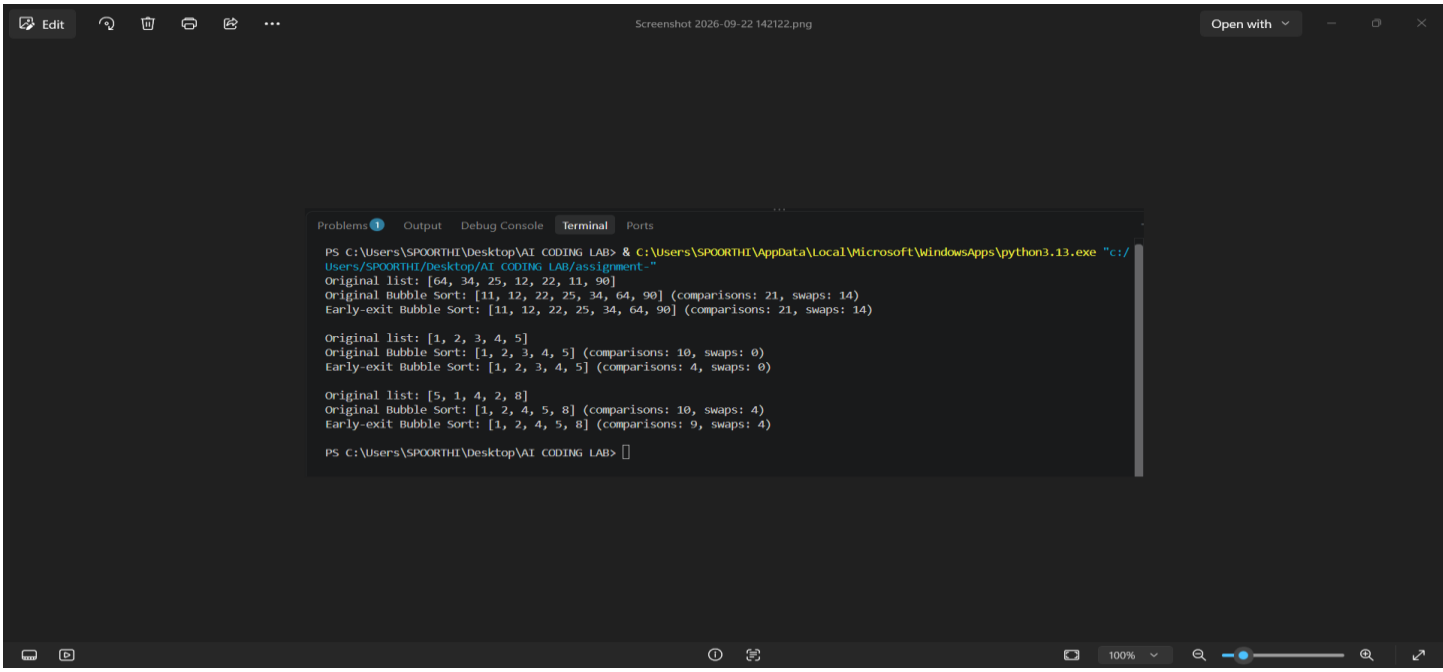
    for pass_number in range(len(sorted_numbers) - 1):
        for index in range(len(sorted_numbers) - 1 - pass_number):
            comparisons += 1
            if sorted_numbers[index] > sorted_numbers[index + 1]:
                sorted_numbers[index], sorted_numbers[index + 1] = (
                    sorted_numbers[index + 1],
                    sorted_numbers[index],
                )
                swaps += 1

    return sorted_numbers, comparisons, swaps


def bubble_sort_early_exit(numbers):
    """Bubble Sort with a shrinking range and early-exit optimization."""
    sorted_numbers = numbers.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(sorted_numbers) - 1):
```

OUTPUT:



OBSERVATION:

Observation

* Bubble Sort compares adjacent elements and swaps them.

* The list is sorted in ascending order.

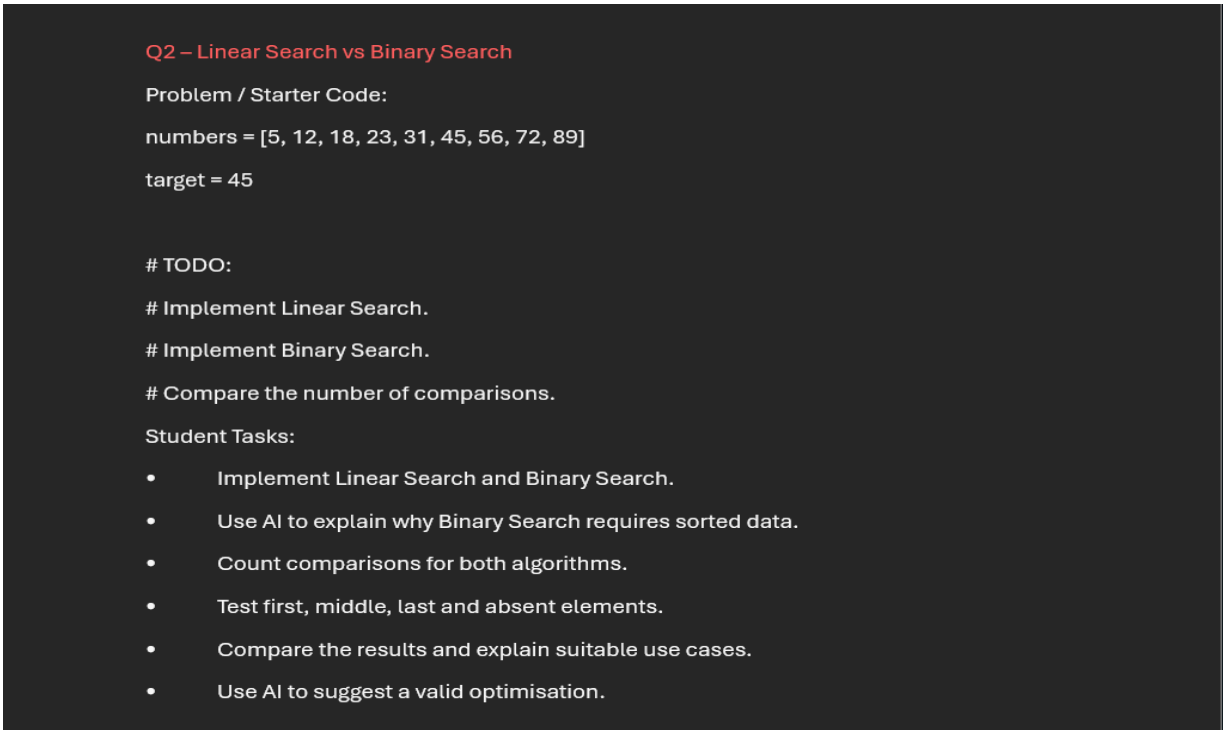
* Each pass moves the largest element to the end.

* Comparisons and swaps depend on the input list.

* Early-exit stops sorting when no swaps occur.

* Early-exit makes Bubble Sort faster for sorted or nearly sorted lists.

QUESTION-2


Q2 – Linear Search vs Binary Search

Problem / Starter Code:

numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]

target = 45

TODO:

Implement Linear Search.

Implement Binary Search.

Compare the number of comparisons.

Student Tasks:

- Implement Linear Search and Binary Search.
- Use AI to explain why Binary Search requires sorted data.
- Count comparisons for both algorithms.
- Test first, middle, last and absent elements.
- Compare the results and explain suitable use cases.
- Use AI to suggest a valid optimisation.

PROMPT:

Implement Linear Search and Binary Search for the given list, count and compare their comparisons for first, middle, last, and absent elements, explain why Binary Search requires sorted data, compare suitable use cases, and suggest one valid optimization.

CODE:

```python
numbers = [5, 12, 18, 23, 31, 45, 56, 72, 89]


def linear_search(numbers, target):
    """Return the target index and number of comparisons."""
    comparisons = 0

    for index, number in enumerate(numbers):
        comparisons += 1
        if number == target:
            return index, comparisons

    return -1, comparisons


def binary_search(numbers, target):
    """Return the target index and number of comparisons."""
    left = 0
    right = len(numbers) - 1
    comparisons = 0

    while left <= right:
        middle = (left + right) // 2
        comparisons += 1

        if numbers[middle] == target:
            return middle, comparisons
        if numbers[middle] < target:
            left = middle + 1
        else:
            right = middle - 1
```

OUTPUT:

```
PS C:\Users\SPOORTHI\Desktop\AI CODING LAB> & C:\Users\SPOORTHI\AppData\Local\Microsoft\WindowsApps\python3.13.exe "c:/
Users/SPOORTHI/Desktop/AI CODING LAB/Assign-2"
First element (5):
  Linear Search: index 0, 1 comparisons
  Binary Search: index 0, 3 comparisons
Middle element (31):
  Linear Search: index 4, 5 comparisons
  Binary Search: index 4, 1 comparisons
Last element (89):
  Linear Search: index 8, 9 comparisons
  Binary Search: index 8, 4 comparisons
Absent element (50):
  Linear Search: index -1, 9 comparisons
  Binary Search: index -1, 3 comparisons

Explanation:
Binary Search requires sorted data because it compares the target with the middle value and discards the half that cann
ot contain it. Without sorted order, that decision could discard the target.
Linear Search works on sorted or unsorted lists and is suitable for small or frequently changing data. Binary Search is
 suitable for large, sorted lists because it runs in O(log n) time instead of Linear Search's O(n) time.
Valid optimization: keep a list sorted when it will be searched repeatedly, then use Binary Search. For one search, sor
ting first may cost more than using Linear Search.
PS C:\Users\SPOORTHI\Desktop\AI CODING LAB>
```

OBSERVATION:

* Linear Search checks elements one by one and works on sorted or unsorted data.

* Binary Search is faster but requires the list to be sorted.

* Binary Search uses fewer comparisons for large datasets.

* For small or unsorted lists, Linear Search is simple and suitable.

QUESTION-3

Q3 – Compare Sorting Algorithms

Problem / Starter Code:

data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]

TODO:

Sort using Bubble Sort, Selection Sort and Insertion Sort.

Compare their performance.

Student Tasks:

- Implement all three sorting algorithms.
- Use the same input for each algorithm.
- Count comparisons and swaps/shifts.
- Use AI to explain strengths and weaknesses.
- Ask AI which algorithm suits nearly sorted data.
- Compare experimental results with theoretical complexity.

PROMPT:

Implement Bubble Sort, Selection Sort, and Insertion Sort for the given data, count comparisons and swaps/shifts, compare their strengths, weaknesses, time complexities, and identify which algorithm is best for nearly sorted data.

CODE:

ASSIGN-3 > data

```python
data = [45, 12, 78, 34, 23, 89, 5, 67, 10, 56]


def bubble_sort(values):
    numbers = values.copy()
    comparisons = 0
    swaps = 0

    for pass_number in range(len(numbers) - 1):
        swapped = False

        for index in range(len(numbers) - 1 - pass_number):
            comparisons += 1
            if numbers[index] > numbers[index + 1]:
                numbers[index], numbers[index + 1] = (
                    numbers[index + 1], numbers[index]
                )
                swaps += 1
                swapped = True

        if not swapped:
            break

    return numbers, comparisons, swaps


def selection_sort(values):
    numbers = values.copy()
    comparisons = 0
    swaps = 0

    for start in range(len(numbers) - 1):
```

OUTPUT:

```
Experimental results using the same input:
Bubble Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
  Comparisons: 44, Swaps: 24
Selection Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
  Comparisons: 45, Swaps: 6
Insertion Sort: [5, 10, 12, 23, 34, 45, 56, 67, 78, 89]
  Comparisons: 31, Shifts: 24
```

OBSERVATIONS:

- All three algorithms correctly sort the same input data in ascending order.
- Bubble Sort and Selection Sort generally require more comparisons.
- Insertion Sort performs well when the data is nearly sorted.
- All three have **$O(n^2)$** average/worst-case time complexity.
- The practical performance depends on the input order and number of swaps/shifts.

QUESTION-4

Optimising a Student Search and Sort Program

Problem / Starter Code:

```
students = [
   ("Arun", 82),
   ("Bala", 65),
   ("Chitra", 91),
   ("Divya", 74),
   ("Esha", 88)
]


# TODO:
# Find the highest mark.
# Sort students by mark.
# Search for a student by name.
# Improve the solution using AI.
```

Student Tasks:

- Write functions for highest mark, sorting and name search.
- Create a straightforward solution first.
- Use AI to identify unnecessary operations.
- Ask AI for a more efficient approach.
- Implement only suggestions you understand.
- Compare original and improved time complexity.

PROMPT:

Implement functions to find the highest mark, sort students by mark, and search by name, then analyze the straightforward solution, identify unnecessary operations, suggest an efficient approach using AI, and compare the original and improved time complexities.

CODE:

```python
students = [
    ("Arun", 82),
    ("Bala", 65),
    ("Chitra", 91),
    ("Divya", 74),
    ("Esha", 88),
]


# Straightforward solution
def highest_mark_basic(students):
    sorted_students = sorted(students, key=lambda student: student[1], reverse=True)
    return sorted_students[0]


def sort_students_basic(students):
    return sorted(students, key=lambda student: student[1], reverse=True)


def search_by_name_basic(students, name):
    for student_name, mark in students:
        if student_name == name:
            return student_name, mark
    return None


# Improved solution
def highest_mark_improved(students):
```

OUTPUT:

```
PS C:\Users\SPOORTHI\Desktop\AI CODING LAB> & C:\Users\SPOORTHI\AppData\Local\Microsoft\WindowsApps
Users/SPOORTHI/Desktop/AI CODING LAB/ASSIGN-4"
Straightforward solution:
Highest mark: ('Chitra', 91)
Sorted students: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Search for Chitra: ('Chitra', 91)

Improved solution:
Highest mark: ('Chitra', 91)
Sorted students: [('Chitra', 91), ('Esha', 88), ('Arun', 82), ('Divya', 74), ('Bala', 65)]
Search for Chitra: ('Chitra', 91)

Analysis:
```

OBSERVATION:

Observation

- The program successfully finds the highest mark, sorts students, and searches by name.
- The straightforward approach may repeat some operations unnecessarily.
- Using efficient searching and sorting methods can reduce execution time.

- The improved solution has better performance for larger student datasets.

QUESTION-5

Q5 – Optimisation Challenge: Best Combination

Problem / Starter Code:

```
items = [
  ("Laptop", 60000),
  ("Phone", 30000),
  ("Tablet", 20000),
  ("Watch", 10000),
  ("Headphones", 5000)
]
budget = 50000
# TODO:
# Find a combination of items whose total price
# is as high as possible without exceeding the budget.
```

Student Tasks:

- Design a solution to maximise total value without exceeding the budget.
- Start with a simple approach and explain it.
- Use AI to compare Brute Force, Greedy and Dynamic Programming.
- Determine which approach guarantees an optimal solution for this formulation.
- Implement the selected solution.
- Test at least three budgets.
- Explain time

PROMPT: Solve the item combination problem using Brute Force, Greedy, and Dynamic Programming, compare their approaches and time complexities, identify which guarantees an optimal solution, implement the best approach, and test it with at least three different budgets.

CODE:

```python
items = [
    ("Laptop", 60000),
    ("Phone", 30000),
    ("Tablet", 20000),
    ("Watch", 10000),
    ("Headphones", 5000),
]

budget = 50000


def brute_force(items, budget):
    """Try every combination and return the most valuable valid one."""
    best_value = 0
    best_items = []

    def search(index, chosen_items, total):
        nonlocal best_value, best_items

        if total > budget:
            return
        if total > best_value:
            best_value = total
            best_items = chosen_items.copy()
        if index == len(items):
            return

        name, price = items[index]
        search(index + 1, chosen_items + [(name, price)], total + price)
        search(index + 1, chosen_items, total)

    search(0, [], 0)
    return best_items, best_value
```

OUTPUT:

```
Budget: 50000
Brute Force: ['Phone', 'Tablet'] -> total = 50000
Greedy: ['Phone', 'Tablet'] -> total = 50000
Dynamic Programming: ['Phone', 'Tablet'] -> total = 50000

Budget: 35000
Brute Force: ['Phone', 'Headphones'] -> total = 35000
Greedy: ['Phone', 'Headphones'] -> total = 35000
Dynamic Programming: ['Phone', 'Headphones'] -> total = 35000

Budget: 70000
Brute Force: ['Laptop', 'Watch'] -> total = 70000
Greedy: ['Laptop', 'Watch'] -> total = 70000
Dynamic Programming: ['Laptop', 'Watch'] -> total = 70000
```

OBSERVATION:

* The program finds the combination of items with the maximum total price without exceeding the budget.

* Brute Force checks all possible combinations and guarantees an optimal solution.

* Greedy is faster but does not always guarantee the optimal combination.

* Dynamic Programming can efficiently find an optimal solution for larger inputs.

* The best approach depends on the number of items and problem constraints.

SSS